

Validação Atômica Não-Bloqueante com Faltas Bizantinas

Aldelir Fernando Luiz^{1,4}, Lau Cheuk Lung², Miguel Correia³,
Valdir Stumm Júnior⁴

¹Departamento de Automação e Sistemas – Universidade Federal de Santa Catarina

²Departamento de Informática e Estatística – Universidade Federal de Santa Catarina

³Instituto Superior Técnico / INESC-ID – Universidade de Lisboa – Portugal

⁴Campus Blumenau – Instituto Federal de Educação, Ciência e Tecnologia Catarinense

aldelir.luiz@posgrad.ufsc.br, lau.lung@ufsc.br,
miguel.p.correia@tecnico.ulisboa.pt, valdir.stumm@blumenau.ifc.edu.br

Abstract. *In this work we present the Byzantine Fault-Tolerant Non-Blocking Weak Atomic Commitment (BFT-NBWAC), a new proposal to solve the Non-Blocking Atomic Commitment (NBAC) problem, where the participants of protocol may fail in a Byzantine way. Our approach is based on virtualization technology, in which we specify a distributed system's architecture that makes possible the detection, masking and confining the faulty behavior of the transactions' participants.*

Resumo. *Neste trabalho apresentamos o BFT-NBWAC (Byzantine Fault-Tolerant Non-Blocking Weak Atomic Commitment), uma proposta inovadora para resolver um problema de acordo conhecido na literatura como Validação Atômica Não-Bloqueante (NBAC), em um ambiente onde os participantes de uma transação podem falhar de maneira bizantina. A abordagem é baseada em um conceito novo denominado “máquinas virtuais gêmeas”, em que são exploradas algumas vantagens fornecidas pela tecnologia de virtualização para especificar uma arquitetura de sistema distribuído, na qual é possível detectar, mascarar e confinar o comportamento faltoso exibido pelos participantes da transação.*

1. Introdução

A noção de transações é essencial para a especificação e implementação de sistemas computacionais, para os quais a consistência acerca da execução de um conjunto de operações sobre dados deve ser rigorosamente preservada. Não obstante, assegurar a terminação de uma transação de maneira consistente em um sistema distribuído não é uma tarefa trivial. Para tal, é requerido o estabelecimento de um acordo entre os participantes envolvidos na transação, a fim de validar (do inglês, *commit*) ou anular (do inglês, *abort*) as operações que a compõem (atomicidade), de modo a torná-las permanentes (durabilidade) ou descartá-las do sistema. O problema que define a validação atômica de transações em sistemas distribuídos é conhecido na literatura como **Validação Atômica Não-Bloqueante** (do inglês, *Non-Blocking Atomic Commitment* – NBAC) [Babaoğlu and Toueg 1993].

O NBAC pode ser visto como um **problema de acordo** em sistemas distribuídos [Pease et al. 1980, Hadzilacos 1990, Guerraoui 1995], e como tal, está sujeito às mesmas restrições teóricas e aos aspectos práticos presentes na concepção de sistemas tolerantes a faltas. Assim, ao longo dos anos um grande número de soluções para o NBAC

foram propostas na literatura, algumas no intuito de circunscrever propriedades e ampliar a resiliência a despeito de faltas de parada (p. ex. *crash*) [Guerraoui et al. 1995, Guerraoui 1995, Guerraoui et al. 1996, Park et al. 2013], e outras para prover otimizações [Guerraoui and Schiper 1995, Abdallah and Pucheral 1999, Greve and Narzul 2002]. Um aspecto particularmente interessante destes estudos é que alguns deles demonstraram as condições necessárias para a resolução do NBAC (p.ex. modelos de sistema, ambientes computacionais, modelos de falhas, detectores de falhas, etc.), além de terem evidenciado as situações nas quais o problema é solúvel.

Embora o NBAC seja um problema bem estabelecido na literatura, tanto em cenários com ou sem faltas de parada, até o momento ele permanece sem uma solução em se tratando de faltas bizantinas [Lamport et al. 1982]. A impossibilidade desta advém de um importante resultado teórico, o qual demonstrou que os problemas de acordo baseados na propriedade de **acordo uniforme** não podem ser resolvidos com faltas bizantinas [Charron-Bost and Schiper 2004]. Isto decorre do simples fato de que o modelo bizantino não assegura a decisão para um processo faltoso, pois nada é previsível a respeito do comportamento bizantino. Este resultado é estendido para outros problemas de acordo especificados a partir da propriedade **uniforme**, como é o caso do NBAC. Particularmente, o NBAC não é solúvel porque mesmo que um participante bizantino reúna as condições necessárias para validar uma transação, ele pode optar por anulá-la propositadamente para deturpar o protocolo. Ou ainda, se o protocolo decidir pela validação, não se pode garantir que o processo bizantino realizará as ações pós-decisão (p.ex. tornar as escritas permanentes).

Neste sentido, este trabalho apresenta uma abordagem prática para resolver a **Validação Atômica Não-Bloqueante** em um ambiente onde os participantes de uma transação podem falhar de maneira bizantina. Para isso, a solução explora uma arquitetura de sistema distribuído baseada em um modelo de falhas híbrido e realista, na qual é realizada uma separação dos participantes da transação em ativos e passivos. Esta separação é realizada a fim de prover os requisitos e as condições necessárias para assegurar as propriedades do NBAC, a despeito de faltas dos participantes. No caso, os participantes ativos (que conduzem o NBAC) podem falhar de maneira bizantina, enquanto os passivos falham apenas por parada. Ao nosso conhecimento, este é o primeiro trabalho da literatura a abordar o NBAC com um modelo híbrido de falhas, de tal maneira que os participantes ativos da transação possam falhar de maneira bizantina. Deste modo, o presente trabalho representa uma contribuição no âmbito das áreas de algoritmos distribuídos tolerantes a faltas, e de arquiteturas e ambientes para a concepção de sistemas distribuídos tolerante a faltas.

O restante deste artigo está organizado da seguinte maneira: a Seção 2 apresenta uma breve descrição do problema da validação atômica não-bloqueante em sistemas distribuídos; a Seção 3 apresenta algumas definições e conceitos preliminares; na Seção 4 é apresentada a solução proposta por este trabalho; a Seção 5 descreve alguns resultados obtidos; e por fim, a Seção 6 descreve as conclusões acerca do trabalho.

2. Caracterização do Problema da Validação Atômica Não-Bloqueante

Tipicamente, uma transação distribuída acessa dados residentes em vários locais (p. ex. sítios) e envolve um conjunto de processos - os participantes, que cooperam entre si para a execução das operações que compõem a unidade lógica de processamento nos sítios [Babaoğlu and Toueg 1993]. Usualmente, um participante desempenha dois papéis em seu local de atuação (ou sítio); o de **gestor de transação** (*Transaction Manager* – TM) e o

de **gestor de dados** (*Data Manager* – DM) - também denominado por **gestor de recursos** (*Resource Manager* – RM) [Babaoğlu and Toueg 1993]. O TM tem como atribuições receber a(s) operação(ões) que lhe foi(ram) solicitada(s), e realizar a cooperação com os demais sítios através de seus TMs, no decurso da transação. Já o DM é o responsável por executar a(s) operação(ões) sobre o(s) dado(s) a que ela(s) faz(em) referência naquele sítio [Babaoğlu and Toueg 1993].

Ao término da transação distribuída, a atomicidade acerca das operações realizadas depende de um acordo entre os participantes engajados na transação. Não obstante, eles devem decidir de maneira única e irrevogável sobre o resultado final da transação, que será pela sua validação (*COMMIT*) - se tudo ocorreu conforme o esperado, ou pela anulação (*ABORT*) - caso algo anormal tenha ocorrido. Este acordo é obtido através de um protocolo de validação atômica, que tem por finalidade assegurar a concordância de todos os participantes quanto ao resultado final da transação, o qual será determinado de acordo com as condições locais de cada participante. Com isso, ao concluir o processamento da transação, cada participante emite um voto (SIM ou NÃO), que declara a sua capacidade em garantir a consistência e a durabilidade da sua parte na transação. E a partir dos votos recebidos é calculado um resultado, que normalmente é *COMMIT* se todos votaram SIM ou, do contrário, o resultado é *ABORT*. Formalmente, a problema da validação atômica não-bloqueante (NBAC) é definido pelas propriedades de [Babaoğlu and Toueg 1993, Guerraoui 1995]:

- **Acordo Uniforme:** todos os processos que decidem obtêm um mesmo valor de decisão;
- **Validade Uniforme:** se um participante decide *COMMIT*, então todos os participantes votaram SIM;
- **Integridade:** um participante não pode reverter sua decisão após ela ter sido tomada;
- **Terminação:** todos os participantes corretos chegam a um valor de decisão;
- **Não-Trivialidade:** se todos os participantes votaram SIM e não há falhas, então a decisão é *COMMIT*.

Rachid Guerraoui em um de seus trabalhos obteve um importante resultado teórico, no qual demonstrou que não é possível resolver o NBAC em ambientes sujeitos a uma simples falta de parada [Guerraoui 1995]. Tal resultado decorre da propriedade **Não-Trivialidade**, que não pode ser plenamente satisfeita na maioria dos sistemas distribuídos que admitem falhas [Guerraoui 1995], isto é, em razão das falhas não poderem ser verificadas de maneira segura mesmo com o uso de detectores de falhas não confiáveis [Chandra and Toueg 1996]. Com isso, o autor definiu uma nova propriedade com exigências mais fracas, a qual deu origem a um novo problema denominado **Validação Atômica Fraca Não-Bloqueante** (ou *Non-Blocking Weak Atomic Commitment* - NB-WAC). O NB-WAC é especificado a partir das mesmas propriedades do NBAC, com exceção da **Não-Trivialidade**, que é enfraquecida levando em conta aspectos mais adequados, realistas e suficientes para os sistemas transacionais. Assim, ao invés de se basear em “falhas”, considera-se apenas “suspeitas”, e a propriedade passa a ser definida por [Guerraoui 1995]:

- **Não-Trivialidade:** se todos os participantes votaram SIM e não há suspeitas de falhas, então a decisão é *COMMIT*.

Note que, diferente do NBAC [Babaoğlu and Toueg 1993] o NB-WAC [Guerraoui 1995] pode ser reduzido ao consenso, já que os resultados obtidos para o consenso com detectores de falhas não-confiáveis podem ser estendidos ao NB-WAC. Neste

ensajo, apresentamos uma contribuição inédita para resolver o NB-WAC em um cenário onde alguns dos participantes da transação podem falhar de maneira bizantina. Em nossa proposta, devido às atribuições inerentes aos papéis de TM e DM definidos no âmbito do NBAC [Babaoğlu and Toueg 1993], consideramos o TM como participante “ativo” e o DM como participante “passivo”. A partir daí, são especificados: (i) uma arquitetura de sistema, por meio de uma abordagem baseada em tecnologia de virtualização [Smith and Nair 2005]; e (ii) um protocolo que é executado sobre tal arquitetura. Neste modelo/sistema, os sítios são especificados a partir de uma abordagem denominada **máquinas virtuais gêmeas** [Dettoni et al. 2013], onde cada sítio é composto por um conjunto de máquinas virtuais, que passam a atuar como réplicas do participante ativo (o TM); e um sistema anfitrião que passa a atuar apenas como participante passivo (o DM). Tal abordagem previne que o comportamento bizantino exibido por um participante ativo faltoso afete a execução do protocolo, de maneira que apenas as faltas de parada dos sítios são percebidas.

3. Conceitos Preliminares

Nesta seção se apresenta os principais conceitos e aspectos teóricos deste trabalho, os quais fundamentam a proposição da solução para o NB-WAC com faltas bizantinas.

3.1. Visão Geral da Solução Proposta

Para facilitar a compreensão acerca da abordagem adotada para a proposição deste trabalho, na Figura 1 é ilustrada a especificação da arquitetura elaborada para prover os requisitos necessários à resolução da validação atômica fraca não-bloqueante com faltas bizantinas.

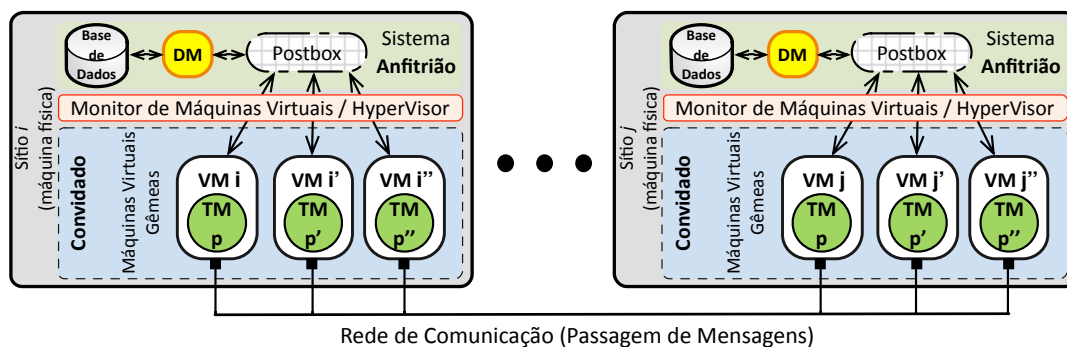


Figura 1. Arquitetura proposta para especificar o protocolo BFT-NBWAC.

Conforme se pode verificar na Figura 1, cada sítio engajado na transação e envolvido com o protocolo BFT-NBWAC é organizado respeitando a especificação original do NBAC (vide Seção 2), onde as entidades TM e DM estão presentes. No entanto, devido a natureza das atribuições definidas para o TM e DM na especificação do NBAC [Babaoğlu and Toueg 1993], neste trabalho define-se o TM como o participante “ativo” da transação e o DM como participante “passivo”. Portanto, em nosso protocolo estes são implementados por processos distintos, que são executados separadamente e em diferentes níveis da arquitetura, diferente de todos os trabalhos verificados na literatura [Guerraoui et al. 1995, Guerraoui and Schiper 1995, Guerraoui 1995, Guerraoui et al. 1996, Abdallah and Pucheral 1999, Park et al. 2013]. Como uma das tarefas do TM é cooperar com os demais sítios engajados na transação, e a do DM é executar as operações sobre os dados, concluímos que não há a necessidade do DM em participar do acordo realizado para a terminação da transação, mas apenas os TMs é que devem fazê-lo.

Sendo assim, no BFT-NBWAC apenas o TM participa do acordo para terminação da transação, sendo executado de maneira independente do DM, que atua isoladamente na

execução das operações da transação (p.ex. escritas e leituras na base de dados). Embora as atribuições do TM e DM sejam complementares, esta separação é um meio factível para resolver o NB-WAC com faltas Bizantinas. E no âmbito deste trabalho, esta separação é concretizada por meio de virtualização [Smith and Nair 2005], em que um sítio é então representado por um **sistema anfitrião** – que implementa o DM e a base de dados utilizada no protocolo, e portanto, pode falhar apenas por parada (*crash*); e por $2f_p + 1$ ¹ **sistemas convidados** – que implementam réplicas de um TM (i.e. o participante ativo) para o sítio e podem falhar de maneira bizantina. Note que, como o NB-WAC não é solúvel em um ambiente puramente bizantino [Charron-Bost and Schiper 2004], a separação proposta permite resolvê-lo a partir de um ambiente onde as falhas manifestadas são híbridas e isoladas.

Em suma, a replicação do TM é um meio factível para mascarar a(s) falta(s) bizantina(s) do participante ativo do BFT-NBWAC que ocorrem em nível de sítio. Assim, o comportamento faltoso de um TM fica confinado ao seu sítio local, e, se manifestado externamente, torna-se facilmente detectável. Na prática isto é realizado com o uso de um esquema de assinaturas de limiar [Shoup 2000], a fim de assegurar a integridade da mensagem e a autenticidade do participante ativo do sítio emissor. Com isso, sempre que uma mensagem tiver de ser enviada para outros sítios, pelo menos $f + 1$ réplicas de TM do emissor devem assinar aquela mensagem a partir do esquema de assinatura de limiar. E se porventura um participante ativo faltoso tentar deturpar o protocolo, o fará isoladamente e será detectado e desconsiderado pelos participantes ativos dos demais sítios envolvidos no protocolo.

3.2. Modelo de Sistema

A arquitetura da Figura 1 é especificada a partir de um modelo híbrido, onde são admitidas diferentes hipóteses acerca do modelo de falhas das entidades que a compõem. O sistema é composto por um universo de processos $\mathcal{U}$, sendo este dividido em alguns subconjuntos. O subconjunto $\mathcal{S} = \{s_1, s_2, \dots, s_n\}$ denota os sítios do sistema distribuído. Cada sítio s_i é composto por um sistema **anfitrião** (*host*), sobre o qual é executado um Monitor de Máquinas Virtuais (ou *Virtual Machine Monitor* – VMM). Para cada sítio s_i , o VMM mantém um conjunto de **máquinas virtuais gêmeas**, que são também conhecidas como sistemas **convidados** (*guests*), tal que $\forall s_i \in \mathcal{S} : \exists \mathcal{G}_{s_i} = \{g_1, g_2, \dots, g_n\}$. O subconjunto $\mathcal{S}$ tem cardinalidade superior a f_s , logo $|\mathcal{S}| \geq f_s + 1$; e os subconjuntos $\mathcal{G}_{s_i}$ têm cardinalidade $\forall s_i \in \mathcal{S} : |\mathcal{G}_{s_i}| \geq 2f_p + 1$.

Cada sistema **anfitrião** mantém uma base de dados, a qual é utilizada nas transações, sendo também o responsável por executar o processo que atua como gestor de dados (DM) no protocolo NB-WAC. Não obstante, o anfitrião mantém um monitor de máquinas virtuais, sobre o qual são executadas como sistemas **convidados** as máquinas virtuais gêmeas. Cada máquina virtual gêmea executa apenas uma réplica do processo que atua no papel de gestor da transação (TM) daquele sítio, de modo que a atuação destas réplicas se dá de maneira conjunta, para representar um único TM em cada sítio. Devido a separação das entidades em níveis distintos da arquitetura, assumimos que: (i) até $f_p \leq \lceil \frac{n-1}{2} \rceil$ máquinas virtuais gêmeas, **por sistema anfitrião** (no caso, os TMs), podem desviar arbitrariamente de suas especificações e então falhar de maneira bizantina; (ii) o sistema anfitrião pode falhar apenas por parada (*crash*), sendo que não mais de f_s anfitriões (i.e. sítios) falham. Assumimos que o sistema anfitrião pode apresentar vulnerabilidades, porém, o VMM fornece isolamento entre os sistemas anfitrião e convidado(s), de tal forma que vulnerabilida-

¹O termos f_s e f_p denotam o limite de faltas dos sítios e dos participantes de um sítio, respectivamente.

des não podem ser exploradas através das máquinas virtuais, nem entre máquinas virtuais [Popek and Goldberg 1974].

Consideramos a existência de um esquema criptográfico de assinaturas de limiar (n, k) [Shoup 2000] baseado no algoritmo RSA [Rivest et al. 1978], onde existem n chaves parciais (ou segredos) e n chaves de verificação, de tal forma que pela combinação de pelo menos k segredos é possível gerar uma assinatura RSA, verificável a partir da chave pública correspondente. Assumimos que o sistema anfitrião - que não é suscetível a faltas bizantinas, atua como distribuidor dos segredos aos processos TMs, sendo que ele gera uma chave privada, a qual fica em sua posse e é dividida em n segredos que são distribuídos a cada uma das máquinas gêmeas, e uma chave pública. A chave pública é usada para verificar a autenticidade e integridade das mensagens assinadas, tanto por meio da chave privada usada pelo sistema anfitrião (p.ex. o DM), como daquelas geradas a partir do esquema de limiar.

A respeito das hipóteses temporais, assumimos o modelo baseado no sincronismo terminal (*eventually synchronous system*) [Dwork et al. 1988], devido às características realistas do mesmo, isto é, ele é assíncrono em grande parte do tempo, mas há períodos de estabilidade, nos quais os tempos são limitados, porém desconhecidos. O protocolo se baseia no uso de temporizadores para detectar falhas de parada dos sítios. Porém, os relógios não são sincronizados, embora os desvios entre eles sejam pequenos - uma suposição válida, considerando as tecnologias atuais. O sistema anfitrião fornece abstrações de memória compartilhada para comunicação entre os processos de um sítio, estas denominadas por *PostBox* [Stumm Júnior et al. 2010]. Na *PostBox* as escritas são *append-only* e as leituras seguem a ordem *FIFO*. Um processo pode escrever apenas em sua *PostBox*, tendo o direito de leitura nas *PostBoxes* dos demais processos do mesmo sítio. A comunicação entre os processos é realizada de duas maneiras: (i) entre as máquinas virtuais gêmeas de um sítio (*intra-sítio*); e (ii) entre os sítios (*inter-sítio*). As comunicações *inter-sítio* ocorrem via canais ponto-a-ponto confiáveis e autenticados, e a comunicação *intra-sítio* ocorre através das *PostBoxes*.

4. Validação Atômica Fraca Não-Bloqueante tolerante a Faltas bizantinas

Esta seção descreve a integração da arquitetura apresentada na Seção 3 (vide Figura 1) e sua relação com a proposição do protocolo que resolve o problema da NB-WAC com participantes ativos sujeitos a faltas bizantinas.

4.1. Princípio de Funcionamento do BFT-NBWAC

A definição dos problemas NBAC e NB-WAC [Babaoğlu and Toueg 1993, Guerraoui 1995] frisam a terminação, e não a execução a transação. Logo, o protocolo apresentado nesta seção é um protocolo para a terminação de transações distribuídas. Por esta razão, consideramos a existência de um suporte de nível superior para a execução da transação, de tal forma que a terminação da transação é realizada por meio do protocolo BFT-NBWAC. A Figura 2 ilustra as fases e os passos de comunicação realizados pelo protocolo BFT-NBWAC, onde o problema é resolvido a partir de três fases e quatro passos de comunicação *inter-sítio*. Note que toda comunicação *intra-sítio* é precedida por uma comunicação *inter-sítio* - esta última ocorre entre as réplicas de TM de um sítio, e entre os TMs e DM do mesmo sítio, ambas a partir das *PostBoxes*.

No caso da Figura 2, o sítio S_1 atua como o líder da transação e é o responsável por conduzir o protocolo de terminação. Na mesma figura, $f_p = 1$, onde cada sítio mantém $2f_p + 1$ réplicas de TM - ali ilustrados por p, p' e p'' . Na primeira fase, o líder solicita a declaração

de voto dos demais participantes sobre validar as operações realizadas na transação, a fim de verificar se eles estão preparados para tal. Na fase seguinte, os sítios não-líderes votam de acordo com a condição em que se encontram, e enviam seus votos ao sítio líder. O voto de um sítio é: *SIM* - se ele está pronto para validar as atualizações; ou *NÃO* caso não esteja pronto ou suspeito que houve falha do líder. Ainda nesta fase, o líder coleta os votos recebidos, avança para a fase três e calcula uma estimativa de resultado para a transação, baseada nas declarações recebidas dos sítios, incluindo a sua. Na sequência, o líder envia a estimativa aos demais sítios, os quais após o recebimento definem seus resultados locais a partir desta. Finalmente, todos os sítios enviam os resultados através de difusão confiável (*reliable multicast*) para subsidiar a decisão pela validação ou anulação da transação.

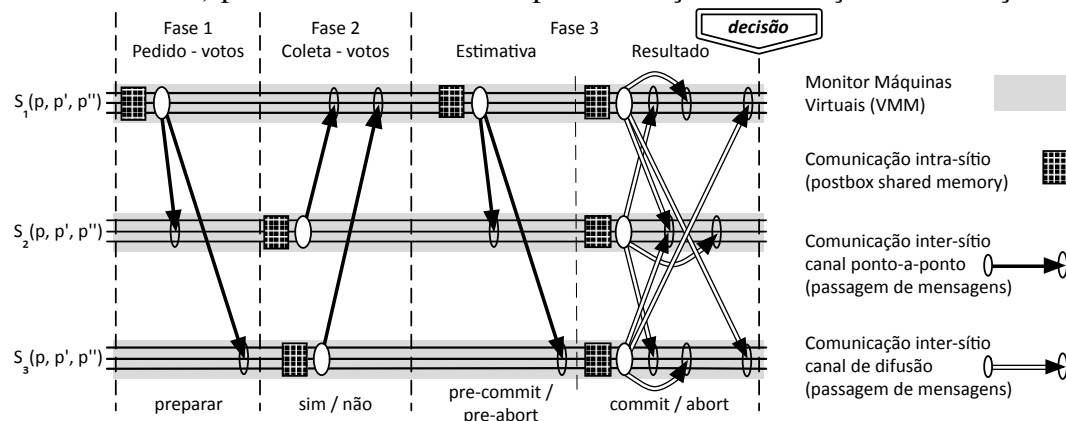


Figura 2. Fases e passos do protocolo BFT-NBWAC.

Note que todas as fases da Figura 2 são necessárias para respeitar as condições e propriedades do NB-WAC (vide Seção 2), onde uma decisão pela validação é legítima apenas se **todos** os sítios votaram *sim*. E para não permitir equívocos na tomada de decisão, o protocolo se orienta da seguinte maneira: (i) na fase 1, se um sítio não puder validar ou não tiver recebido o pedido de voto do líder em um tempo o suficiente, ele vota *não*; do contrário, ele vota *sim*; (ii) na fase 2, se porventura o líder não receber **todos** os votos dos sítios não-líderes, ele calculará a estimativa como *pre-abort*; ou do contrário, como *pre-commit*; (iii) na fase 3, o resultado será subsidiado pela estimativa recebida do líder, sendo que, caso algum sítio não receba a estimativa e suspeite de falha no líder, ele determinará seu resultado local como *abort*, o que incorrerá na anulação da transação pelo protocolo. Em todos os casos, observa-se que uma decisão pela validação ocorrerá somente se todos os sítios votaram *sim*, e em cada fase do protocolo não houve suspeita(s) de falha(s).

4.2. Base Algorítmica do BFT-NBWAC

Esta seção apresenta os detalhes do protocolo BFT-NBWAC, que é especificado a partir dos algoritmos das Figuras 3, 4 e 5. Cabe ressaltar que estes algoritmos descrevem apenas a implementação dos participantes ativos do NB-WAC (i.e., TMs), que compreendem as entidades responsáveis pela execução do protocolo de terminação [Babaoğlu and Toueg 1993]. Neste caso, o participante passivo (o DM) é apenas consultado pelos TMs para verificar se as operações da transação foram completamente executadas, e sem erros. O algoritmo da Figura 3 descreve as variáveis utilizadas pelo protocolo e também a tarefa principal do BFT-NBWAC, que é invocada pelos sítios participantes ao término da transação. Esta tarefa especifica o protocolo que coordena a terminação da transação, cujo propósito é obter um resultado final único e uniforme para a transação, a despeito da ocorrência de faltas bizantinas em alguns dos participantes. É digno de nota que o algoritmo que especifica a comunicação *intra-sítio* entre os TMs, e entre TMs e DM, foi omitido devido a restrições de espaço.

Os algoritmos das Figuras 3, 4 e 5 referenciam o algoritmo que implementa esta comunicação, a partir da função *intrasite_validate*. Note que sempre que há a necessidade de um sítio em se comunicar com outros sítios, esta função é invocada naqueles algoritmos. Ademais, esta função também implementa um esquema de assinaturas de limiar [Shoup 2000], que é usado para prover a integridade e autenticidade das mensagens oriundas dos participantes ativos de um sítio, sendo essencial para o funcionamento do protocolo. Além das garantias mencionadas, o uso deste esquema impede um processo faltoso de forjar assinaturas sobre mensagens espúrias. E se porventura um processo faltoso insistir no envio de mensagens inválidas, elas serão descartadas no receptor, pois toda a recepção ou entrega (para difusão confiável) de mensagens no protocolo é condicionada à verificação da assinatura, a partir da chave pública do sítio emissor (vide algoritmos das Figuras 3, 4 e 5).

```

Declaration of Variables:
1:  $\mathcal{S} = \{S_1, S_2, \dots, S_n\}$  /* Sítios que participam das transações distribuídas */
2:  $\mathcal{P}[|\mathcal{S}|] = \emptyset$  /* Processos gêmeos de cada sítio  $S_i$  */
3:  $vote : \{yes, no\}$  /* Votos possíveis para a terminação de uma transação */
4:  $outcome : \{commit, abort\}$  /* Resultados possíveis para a terminação do protocolo */
5:  $estimate : \{pre-commit, pre-abort\}$  /* Estimativas possíveis para uma decisão */

Local variables: /* Variáveis inicializadas a cada rodada do protocolo */
6:  $task \leftarrow \perp$ 
7:  $vote \leftarrow \perp$ 
8:  $estimate \leftarrow \perp$ 
9:  $outcome \leftarrow \perp$ 
10:  $decision \leftarrow \perp$ 
11:  $leader \leftarrow \perp$ 

procedure Atomic.Commit( $T_{id}$ ) /* Procedimento que inicia o protocolo de validação atômica não-bloqueante */
12:  $leader \leftarrow S_k : k = T_{id} \bmod |\mathcal{S}|$  /* Define o coordenador para conduzir o protocolo BFT-NBWAC */
13: if  $p_j \in leader$  then
14:    $task \leftarrow TaskLeader$  /* Se for o coordenador, inicia a tarefa de coordenador */
15:   fork  $task$ 
16: else
17:    $task \leftarrow TaskNonLeader$  /* Se não for o coordenador, inicia a tarefa normal */
18:   fork  $task$ 
19:  $task.join()$  /* Todos os processos aguardam o término da tarefa iniciada de acordo com o papel do sítio */

/* Phase 3 : Outcome */
20: if  $decision = \perp$  then
21:    $\langle S_i, OUTCOME, T_{id}, outcome \rangle \leftarrow partial\_sign(p_j, \langle S_i, OUTCOME, T_{id}, outcome \rangle)$ 
22:    $\langle S_i, OUTCOME, T_{id}, outcome \rangle \leftarrow intrasite\_validate(\langle S_i, OUTCOME, T_{id}, outcome \rangle_\sigma)$ 
23:    $R\_multicast((\bigcup_{S_j \in \mathcal{S}} \mathcal{P}[S_j]), \langle S_i, OUTCOME, T_{id}, outcome \rangle_{\sigma_{f+1}})$ 
24:   wait until  $[(R\_deliver(\langle S_i, OUTCOME, T_{id}, outcome \rangle_{\sigma_{f+1}}) \text{ from each } S_j \in \mathcal{S} : \text{verifysig}(S_j, \langle S_i, OUTCOME, T_{id}, outcome \rangle_{\sigma_{f+1}})) \text{ or } (\Delta_{S_j} \text{ expires})]$  /* Decide */

25:    $\forall S_j \in \mathcal{S} \text{ such that } \langle -, OUTCOME, T_{id}, - \rangle_{\sigma_{f+1}} \text{ was delivered} \Rightarrow$ 
      $proposal[S_j] \leftarrow \{delivered(\langle S_j, OUTCOME, T_{id}, outcome \rangle) \text{ from } S_j\}$ 
26:   if  $(|\bigcup_{S_j \in \mathcal{S}} proposal[S_j]| = |\mathcal{S}|) \wedge (\nexists propose \in (\bigcup_{S_j \in \mathcal{S}} proposal[S_j]) : \forall S_j \Rightarrow proposal[S_j] = abort)$  then
27:      $decision \leftarrow commit$ 
28:   else
29:      $decision \leftarrow abort$ 
30:    $decide(S_i, decision)$ 

upon  $R\_deliver(\langle S_i, DECISION, T_{id}, abort \rangle_{\sigma_{f+1}})$  from some  $p_k$  of  $S_j \in \mathcal{S} : verifysig(S_j, \langle S_i, DECISION, T_{id}, abort \rangle_{\sigma_{f+1}})$ 
31:  $decision \leftarrow abort$  /* Decisão antecipada caso algum sítio tenha declarado o voto como não */
32:  $task.terminate()$  /* Encerra a tarefa iniciada de acordo com o papel do participante */
    
```

Figura 3. Tarefa principal do protocolo BFT-NBWAC – sítio s_i – processo p_j .

O algoritmo especificado pela Figura 3 é iniciado quando um participante conclui sua parte no processamento da transação, onde invoca o procedimento *Atomic.Commit()* para realizar a validação atômica da transação em lide. No caso, um processo p_j que implementa uma réplica de TM do sítio s_i , primeiramente inicializa as variáveis para aquela rodada do protocolo nos valores padrões (linhas 6 a 11), e posteriormente define o líder da transação baseado nos identificadores dos sítios e da transação em questão (linha 12). Se o processo

pertence ao sítio definido como líder, ele inicia uma tarefa para executar as atividades de líder no protocolo BFT-NBWAC (linhas 13 a 15), cujo algoritmo é especificado na Figura 4. De outro modo, se o processo for apenas um participante, ele inicia uma tarefa para realizar as atividades de não-líder no protocolo (linhas 16 a 18), especificado pelo algoritmo da Figura 5. O código entre as linhas 19-30 compreende à última fase do protocolo - a de decisão, a qual é comum a todos os participantes independente do papel exercido. Após o lançamento das tarefas de acordo com os papéis dos sítios, os processos aguardam o término de suas respectivas tarefas e retornam para a tarefa principal (função *task.join()* da linha 19 - Figura 3), a fim de obter uma decisão e concluir a rodada do protocolo. No que segue, o protocolo será explicado de acordo com as fases do protocolo ilustradas pela Figura 2.

```

TaskLeader:                                     /* Apenas o participante líder da transação executa a tarefa - Phase 1 */
1:  $\langle S_i, \text{REQUEST-VOTE}, T_{id} \rangle \leftarrow \text{partial\_sign}(p_j, \langle S_i, \text{REQUEST-VOTE}, T_{id} \rangle)$ 
2:  $\langle S_i, \text{REQUEST-VOTE}, T_{id} \rangle \leftarrow \text{intrasite\_validate}(\langle S_i, \text{REQUEST-VOTE}, T_{id} \rangle_\sigma)$ 
3:  $\text{send}(p_j, \langle S_i, \text{REQUEST-VOTE}, T_{id} \rangle_{\sigma_{f+1}})$  to  $\forall p_k \in ((\bigcup_{S_j \in S} \mathcal{P}[S_j]) \setminus \mathcal{P}[S_i])$                                      /* Phase 2 */

4: wait until  $[(\text{received}(\langle S_j, \text{VOTE}, T_{id}, \text{vote} \rangle_{\sigma_{f+1}})$  from some  $p_k$  of each  $S_j \in (S \setminus \{S_i\}) :$ 
    $\text{verifysig}(S_j, \langle S_j, \text{VOTE}, T_{id}, \text{vote} \rangle_{\sigma_{f+1}}))$  or  $(\Delta_{S_j} \text{ expires})]$ 
5: for each  $S_j \in (S \setminus \{S_i\})$  do
6:    $\text{msgs}[S_j] \leftarrow \{\text{received}(\langle S_j, \text{VOTE}, T_{id}, \text{vote} \rangle_{\sigma_{f+1}})$  from some  $p_j : p_j \in S_j\}$ 
7:    $\text{vote}[S_j] \leftarrow \text{compute\_site\_vote}(\text{msgs}[S_j])$ 
8:  $\text{vote} \leftarrow \text{query\_Data\_Manager}(T_{id})$ 
9:  $\langle S_i, \text{VOTE}, T_{id}, \text{vote} \rangle \leftarrow \text{partial\_sign}(p_j, \langle S_i, \text{VOTE}, T_{id}, \text{vote} \rangle)$ 
10:  $\text{msgs}[S_i] \leftarrow \text{intrasite\_validate}(\langle S_i, \text{VOTE}, T_{id}, \text{vote} \rangle_\sigma)$ 
11:  $\text{vote}[S_i] \leftarrow \text{msgs}[S_i]$                                      /* Phase 3 : Estimate */

12: if  $(|\bigcup_{S_j \in S} \text{vote}[S_j]| = |S|) \wedge (\nexists \text{vote} \in (\bigcup_{S_j \in S} \text{vote}[S_j]) : \forall S_j \Rightarrow \text{vote}[S_j] = \text{no})$  then
13:    $\text{estimate} \leftarrow \text{pre-commit}$ 
14: else
15:    $\text{estimate} \leftarrow \text{pre-abort}$ 
16:  $\langle S_i, \text{ESTIMATE}, T_{id}, \text{estimate}, \text{msgs}[] \rangle \leftarrow \text{partial\_sign}(p_j, \langle S_i, \text{ESTIMATE}, T_{id}, \text{estimate}, \text{msgs}[] \rangle)$ 
17:  $\langle S_i, \text{ESTIMATE}, T_{id}, \text{estimate}, \text{msgs}[] \rangle \leftarrow \text{intrasite\_validate}(\langle S_i, \text{ESTIMATE}, T_{id}, \text{estimate}, \text{msgs}[] \rangle_\sigma)$ 
18:  $\text{send}(p_j, \langle S_i, \text{ESTIMATE}, T_{id}, \text{estimate}, \text{msgs}[] \rangle_{\sigma_{f+1}})$  to  $\forall p_k \in ((\bigcup_{S_j \in S} \mathcal{P}[S_j]) \setminus \mathcal{P}[S_i])$ 
19:  $\text{outcome} \leftarrow (\text{estimate} = \text{pre-commit}) ? \text{commit} : \text{abort}$ 
    
```

Figura 4. Tarefa executada pelos TMs do sítio líder – sítio S_i – processo p_j .

Fase 1 O sítio líder dá início ao protocolo, sendo que os demais sítios aguardam a manifestação do líder para iniciá-lo. Cada processo réplica de TM do sítio líder, quando adentra na primeira fase do protocolo solicita aos participantes dos demais sítios, através de seus respectivos TMs, uma declaração de intenção em validar as operações da transação. Para tanto, cada processo TM do líder envia uma mensagem REQUEST-VOTE assinada através do esquema de limiar, que é gerada por pelo menos $f + 1$ TMs daquele sítio (linhas 1 a 3 - Figura 4). Os processos TMs dos sítios não-líderes - que estão à espera da manifestação do líder (linha 1 - Figura 5), quando recebem uma mensagem REQUEST-VOTE contendo uma assinatura válida do sítio líder ou; de outro modo, quando algum TM de um sítio não-líder não recebe a manifestação inicial do sítio líder em um tempo suficiente, um temporizador é expirado; e após um destes eventos a fase 2 do protocolo é iniciada.

Fase 2 Nesta fase, os TMs líderes aguardam pelos votos solicitados aos TMs não-líderes na fase anterior (linha 4 - Figura 4), enquanto os TMs não-líderes, ao iniciar esta fase do protocolo fazem o seguinte: (i) se o temporizador expirou e não receberam o pedido de voto do sítio líder, eles passam a suspeitar de falha do líder, e declaram seu voto como *não* (linhas 2 e 3 - Figura 5) - note que, isso pode ocorrer apenas se o sítio líder falhar por parada, pois todas as réplicas corretas do TM líder enviaram a mensagem na fase anterior; (ii) se recebem uma mensagem com o pedido de voto assinada corretamente pelo sítio líder, eles consultam ao DM de seu sítio local, a fim de verificar se sua parte da transação logrou êxito (linhas 4 e

5 - Figura 5). Esta consulta ocorre através de comunicação *intra-sítio* entre os TMs e o DM de um mesmo sítio, em que a função *query_Data_Manager* retorna *SIM* se houve êxito; ou *NÃO* caso algo impeça a transação de ser validada. E baseado no retorno do DM, os TMs dos sítios não-líderes preparam suas declarações de voto através da mensagem *VOTE*, com os identificadores do sítio e da transação para a qual o voto está sendo emitido, bem como o voto propriamente dito. A mensagem é submetida ao processo de validação *intra-sítio* já mencionado e, uma vez assinada ela é enviada ao líder (linhas 6 a 9 - Figura 5), e os TMs não-líderes passam a aguardar por uma estimativa de resultado proveniente do sítio líder, para então dar sequência à próxima fase do protocolo (linha 14 - Figura 5).

```

TaskNonLeader:                                     /* Todos os participantes não-líderes da transação executam a tarefa - Phase 1 */
1: wait until [(received( $\langle S_j, \text{REQUEST-VOTE}, T_{id} \rangle_{\sigma_{f+1}}$ ) from some  $p_k$  of  $S_j : S_j = \text{leader} \wedge \text{verifysig}(\text{msg}.S_j, \langle \text{msg} \rangle_{\sigma})$ )
   or ( $\Delta_{S_j}$  expires)]
                                                                                                     /* Phase 2 */
2: if  $\Delta_{S_j}$  expired then
3:    $\text{vote} \leftarrow \text{no}$ 
4: else
5:    $\text{vote} \leftarrow \text{query\_Data\_Manager}(T_{id})$ 
6: if  $\text{vote} = \text{yes}$  then
7:    $\langle S_i, \text{VOTE}, T_{id}, \text{vote} \rangle \leftarrow \text{partial\_sign}(p_j, \langle S_i, \text{VOTE}, T_{id}, \text{vote} \rangle)$ 
8:    $\langle S_i, \text{VOTE}, T_{id}, \text{vote} \rangle \leftarrow \text{intrasite\_validate}(\langle S_i, \text{VOTE}, T_{id}, \text{vote} \rangle_{\sigma})$ 
9:    $\text{send}(p_j, \langle S_i, \text{VOTE}, T_{id}, \text{site\_vote} \rangle_{\sigma_{f+1}})$  to  $\forall p_k \in \text{leader}$ 
10: else
11:    $\langle S_i, \text{DECISION}, T_{id}, \text{abort} \rangle \leftarrow \text{partial\_sign}(p_j, \langle S_i, \text{DECISION}, T_{id}, \text{abort} \rangle)$ 
12:    $\langle S_i, \text{DECISION}, T_{id}, \text{abort} \rangle \leftarrow \text{intrasite\_validate}(\langle S_i, \text{DECISION}, T_{id}, \text{abort} \rangle_{\sigma})$ 
13:    $R\text{-multicast}((\bigcup_{S_j \in S} \mathcal{P}[S_j]), \langle S_i, \text{DECISION}, T_{id}, \text{abort} \rangle_{\sigma_{f+1}})$ 
                                                                                                     /* Phase 3 : Estimate */
14: wait until [(received( $\langle S_j, \text{ESTIMATE}, T_{id}, \text{estimate}, \text{msgs}[] \rangle_{\sigma_{f+1}}$ ) from some  $p_k$  of  $S_j :$ 
    $S_j = \text{leader} \wedge \text{verifysig}(S_j, \langle S_j, \text{ESTIMATE}, T_{id}, \text{estimate}, \text{msgs}[] \rangle_{\sigma_{f+1}})$ ) or ( $\Delta_{S_j}$  expires)]
15: if  $\Delta_{S_j}$  expired then
16:    $\text{outcome} \leftarrow \text{abort}$ 
17: else
18:    $\text{msg} \leftarrow \{ \text{received}(\langle S_j, \text{ESTIMATE}, T_{id}, \text{estimate}, \text{msgs}[] \rangle_{\sigma_{f+1}}) \text{ from } p_k \in \text{leader} \}$ 
19:    $\text{outcome} \leftarrow (\text{msg.estimate} = \text{pre-commit}) ? \text{commit} : \text{abort}$ 
20:   for all  $S_j \in S$  do
21:     if  $\neg(\text{verifysig}(S_j, \langle \text{msg.msgs}[S_j] \rangle)) \vee \neg(\text{compute\_site\_vote}(\text{msg.msgs}[S_j]) = \text{yes})$  then
22:        $\text{outcome} \leftarrow \text{abort}$ 
23:   break

```

Figura 5. Tarefa executada pelos TMs dos sítios não-líderes – sítio s_i – processo p_j .

Um TM do sítio líder, ao receber as mensagens contendo a declaração de voto de cada um dos sítios não-líderes, armazena as mensagens recebidas e computa os respectivos votos (linhas 5 a 7 - Figura 4). E como o TM líder também é participante da transação, ele também consulta seu DM local, para subsidiar sua declaração de voto. Nos mesmos moldes, a mensagem de voto do sítio líder é assinada pelo esquema de limiar pelos TMs líderes (linhas 8 e 9 - Figura 4); que posteriormente armazenam a mensagem assinada junto aos demais votos recebidos (conjuntos $\text{msgs}[S_i]$ e $\text{vote}[S_i]$ – linhas 10 e 11 da Figura 4).

Fase 3 Ao adentrar nesta fase, o primeiro passo dos TMs líderes é verificar se eles receberam os votos de todos os participantes e se dentre os que foram recebidos não houve nenhuma declaração *não* - ambas as condições necessárias para decidir pela validação (linha 12 - Figura 4). Se estas condições forem satisfeitas, é um indício de que **todos** estão de acordo e não houve falhas, e então a transação pode ser validada. Neste caso, cada TM líder calcula a estimativa como *pre-commit*; do contrário, se uma das condições não for satisfeita a estimativa será *pre-abort*. Na sequência, a mensagem de estimativa é validada junto aos demais TMs líderes e posteriormente é enviada aos sítios não-líderes (linhas 16 a 18 - Figura 4). Note que junto à estimativa, o líder envia aos demais sítios todas as mensagens que subsidiaram a mesma. De posse da estimativa, o líder determina o seu resultado para a transação e o guarda para a etapa final do protocolo (linha 19 - Figura 4). De outro modo, os TMs

não-líderes iniciam esta fase em duas situações: quando receberem a estimativa do líder; ou quando do esgotamento do temporizador, caso não tenham recebido a estimativa dentro de um período estabelecido pelo protocolo (linha 14 - Figura 5). A partir do recebimento da estimativa, os TMs não-líderes determinam seus resultados como: *commit* se recebeu *pre-commit*; *abort* se recebeu *pre-abort*. E de maneira conservadora, após definirem os resultados os mesmos TMs verificam as mensagens que resultaram naquela estimativa (p.ex. os votos), as quais vieram junto à mensagem da estimativa (linhas 20 a 23 - Figura 5). Se tal verificação constatar o recebimento de alguma mensagem inválida, ou uma declaração de voto *não*, o resultado será alterado para *abort*. Se porventura o temporizador se esgotar e a estimativa não tiver sido recebida, eles passam a suspeitar de falha do líder e declaram implicitamente seu resultado como *abort* (linhas 15 e 16 - Figura 5).

Quando os TMs de todos os sítios concluem suas tarefas executadas para o protocolo, eles retornam ao algoritmo da tarefa inicial (Figura 3), o qual ficou bloqueado até a conclusão das respectivas tarefas (função *task.join()* da linha 19 - Figura 3). O código das linhas 20 a 30 compreende à última etapa do protocolo, a **decisão**, que é executada por todos os TMs. Primeiramente eles verificam se até aquele ponto não houve nenhuma decisão para a transação, e em não tendo havido, cada TM pega o resultado que obteve na fase anterior e envia aos TMs dos demais sítios através de *difusão confiável* [Hadzilacos and Toueg 1994] (linhas 20 a 23 - Figura 3). Logo após, os TMs de cada sítio aguardam pela entrega das mensagens contendo os resultados obtidos por todos os demais sítios (linha 24 - Figura 3).

Note que a mensagem do resultado de um sítio não será entregue somente no caso do sítio falhar por parada, e antes do envio, pois a mensagem é enviada através de difusão confiável. Ao concluir o passo da linha 24, os TMs verificam se as mensagens dos resultados de cada sítio foram entregues, e se dentre elas não há nenhuma que visa a anulação da transação (linha 26 - Figura 3). Neste caso, uma decisão legítima será pela validação (*commit*), já que ambas as condições necessárias para a validação da transação foram satisfeitas. Do contrário, se uma das condições não for satisfeita a decisão será pela anulação (*abort*). Ao invocar a função *decide*, cada TM dos sítios insere o valor de decisão em sua *PostBox* para orientar o DM a realizar as alterações ou descartá-las da base de dados, que o faz de acordo com a decisão verificada na *PostBox* de pelo menos $f + 1$ TMs daquele sítio. Isto é o que garante a não-reversão da decisão e o acordo uniforme no sítio, a despeito de faltas.

É importante salientar que o protocolo admite decisões antecipadas, tal como previsto na especificação do NBAC [Babaoğlu and Toueg 1993]. Esta admissão é plausível, pois de acordo com as propriedades do NBAC, a anulação pode ser vista como a única decisão aceitável e legítima. Em nosso protocolo tal condição é especificada nas linhas 10 a 13 do algoritmo da Figura 5, onde um participante decide de maneira unilateral pela anulação da transação, se por alguma razão não puder validá-la. No caso, os TMs daquele sítio enviam aos demais sítios uma mensagem *DECISION* através da primitiva de *difusão confiável*, para assegurar a entrega desta pelos demais sítios. A entrega da mensagem é vista nas linhas 31 e 32 da tarefa principal do protocolo (algoritmo da Figura 3), onde os participantes, independentemente do papel que estão a desempenhar, decidem de maneira definitiva pela anulação e encerram as respectivas tarefas que haviam iniciado. Por conseguinte, cada participante retorna para a tarefa principal e efetiva a decisão recebida (linhas 19, 20 e 30 - Figura 3).

Salientamos que as provas de correção para os algoritmos apresentados foram omitidas devido à restrições de espaço. Todavia, as provas e também o algoritmo que especifica a comunicação *intra-sítio* estão disponíveis na versão estendida do artigo [Luiz et al. 2013].

5. Avaliação do Protocolo

No intuito de verificar a eficiência da solução proposta, realizamos uma comparação analítica dos resultados obtidos para o BFT-NBWAC e de alguns protocolos de validação atômica encontrados na literatura. Esta avaliação é realizada nos mesmos moldes dos trabalhos relacionados [Guerraoui and Schiper 1995, Guerraoui et al. 1996, Abdallah and Pucheral 1999, Greve and Narzul 2002], onde as medidas de desempenho verificadas são aquelas recorrentes na literatura, pela análise de cenários favoráveis (p. ex. sem falhas), onde todos os processos votam *sim*. A Tabela 1 descreve os resultados verificados para o BFT-NBWAC, para o clássico 3PC (*Three Phase Commit*) para os trabalhos denominados: DNB-AC [Guerraoui and Schiper 1995], MD3PC [Guerraoui et al. 1996], NB-2PC [Greve and Narzul 2002] e ANB-CLL [Abdallah and Pucheral 1999].

Tabela 1. Comparação entre alguns protocolos de validação atômica.

Protocolo	# Passos	# Mensagens	Resiliência	# Participantes	# Processos	Faltas
3PC	5	$5n$	—	—	—	parada
DNB-AC	3	$2n^2 + n$	$f < n/2$	$2f + 1$	$2f + 1$	parada
MD3PC	3	$3nf + 3n$	$f < n/2$	$2f + 1$	$2f + 1$	parada
NB-2PC	3	$2nf + 3n$	$f < n$	$f + 1$	$f + 1$	parada
ANB-CLL	2	$n^2 + n$	$f < n/2$	$2f + 1$	$2f + 1$	parada
BFT-NBWAC	4	$3(n - 1) + n^2$	$f < n$	$f_s + 1$	$2f_p f_s + 2f_p + f_s + 1$	bizantina

Para os resultados exibidos na Tabela 1, dois pontos merecem destaque. O ANB-CLL obtém a terminação em apenas dois passos, já que ele elimina a fase de votação, por se basear nas propriedades locais dos protocolos de serialização dos participantes, para obter o resultado da transação. Em decorrência disso, o protocolo tem restrições de uso apenas em ambientes onde é possível conhecer a organização local dos participantes. O NB-2PC apresenta a melhor resiliência, em razão dele permitir decisões antecipadas quando o ambiente reúne condições favoráveis para tal. No caso, a decisão é delegada a um subconjunto de processos $Set_{NB} \leq N$ previamente escolhido (p.ex. os mais confiáveis). Notadamente, os protocolos avaliados apresentam maior eficiência em relação ao BFT-NBWAC, primeiro por tolerarem apenas faltas de parada; e segundo, porque todos eles são especificados a partir de um serviço de *consenso* subjacente, onde o acordo é resolvido com o auxílio de um detector de falhas da classe $\diamond S$. E isto implica que as mensagens e passos incidentes do acordo realizado através do *consenso*, não são refletidos no cômputo das medidas destes trabalhos.

Por outro lado, o BFT-NBWAC requer um número menor de participantes do que alguns protocolos avaliados, devido aos demais considerarem o sítio como participante. Em nosso caso, como um participante é implementado no sítio por $2f + 1$ réplicas, um número maior de processos é requerido para tolerar as faltas bizantinas. Um ponto que também merece destaque é que, diferente do que ocorre nos demais trabalhos, onde o custo associado à difusão confiável usada para propagar a decisão do protocolo é omitido, o número de mensagens apresentado para o BFT-NBWAC representa o número total de mensagens, considerando a difusão confiável. Já o número de passos requerido decorre do fato de que em nosso protocolo o acordo é baseado no líder, pois para que os processos possam decidir seguramente é necessário que todos tenham ciência do voto de todos. Se optássemos por não fazer desta maneira, um número maior de mensagens seria requerido para o protocolo.

E finalmente, a fim de realizarmos uma prova de conceito, implementamos um protótipo simplificado do BFT-NBWAC. Para tal, adotamos a linguagem Java e as bibliotecas *ThreshSig* (<http://threshsig.sourceforge.net>) e *Chronicle* (<https://github.com/peter-lawrey/Java-Chronicle>) para implementar o esquema de assinaturas de limiar e as *Post-Boxes*, respectivamente. Para os canais de comunicação, utilizamos os *SocketChannels* dis-

poníveis na API Java NIO. Os experimentos foram realizados em um ambiente de rede local, composto por máquinas com as seguintes configurações: três máquinas com 8GB de RAM; 1 Interface Ethernet Gigabit, sendo 1 (uma) com processador Intel® Core™I5-2410M 2.30GHz; 1 (uma) com processador Intel® Core™I3-2310M 2.10GHz e 1 (uma) com processador Intel® Core™2 Duo P8600 2.40GHz. As máquinas foram interligadas em uma rede cabeada a partir de um roteador Cisco® DPC3925. Em cada uma das máquinas foi instalado o sistema operacional Debian 7.0 ("wheezy") em nível de anfitrião, sobre o qual foram instalados o *VMware Workstation* 10 como VMM e o MySQL 5.5.33 como base de dados. Em cada VMM foram criadas 3 máquinas virtuais (ou $2f + 1$, onde $f = 1$) com o mesmo sistema operacional (Debian 7.0), sendo reservados 1.5GB de RAM para cada uma.

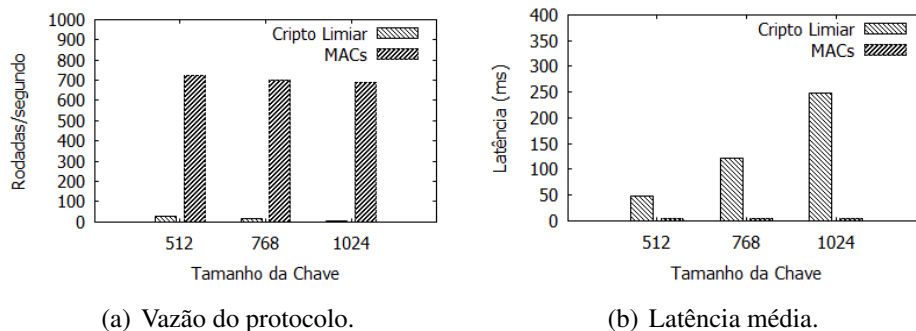


Figura 6. Avaliação experimental do protocolo BFT-NBWAC.

Os experimentos foram realizados a partir da especificação original do BFT-NBWAC e de uma implementação onde o esquema de assinaturas de limiar foi substituído por autenticadores - códigos de autenticação de mensagens (MACs), ambos com variação no tamanho das chaves. O intuito da avaliação foi o de analisar o custo incidente da rodada (*rounds*) do acordo para a terminação da transação. Para tanto, cada sítio foi carregado com um arquivo contendo 10.000 transações identificadas e não concorrentes, a partir das quais as instâncias do acordo foram iniciadas nos sítios, com um intervalo de 10ms. Observamos que o maior impacto no desempenho do BFT-NBWAC foi causado pelo mecanismo de assinaturas utilizado (Figura 6(a)). O uso de assinaturas de limiar incidiu em uma degradação demasiada do desempenho. Conforme os resultados reportados na Figura 6(b), o tempo médio consumido por uma assinatura de limiar é drasticamente maior em ordens de magnitude do que o consumido por códigos de autenticação de mensagens. Em nossos experimentos, o tempo médio consumido para a geração de uma assinatura de limiar em relação aos código de autenticação de mensagens foi de aproximadamente 15 vezes maior para chaves de 512 bits, e de aproximadamente 45 e 60 vezes para chaves de 768 bits e 1024 bits, respectivamente.

6. Considerações Finais

Este artigo apresentou uma arquitetura e um protocolo para resolução do problema da validação atômica não-bloqueante em sistema distribuídos, a fim de oferecer uma solução em cenários onde os participantes da transação podem falhar de maneira bizantina. A solução foi desenvolvida a partir de uma abordagem pragmática e realista, com o uso de tecnologia de virtualização, sendo adotado um modelo híbrido de faltas, com diferentes hipóteses para cada nível da arquitetura do sistema. À primeira vista, acreditamos que os resultados trazidos por este trabalho representam um passo inicial para que soluções práticas, voltadas para ambiente reais, possam vir a ser desenvolvidas. Também, por ser realista e baseada em tecnologia moderna, a ideia proposta pode ser vista como um avanço para a área de algoritmos distribuídos tolerante a faltas. Ademais, a proposta é passível de adaptação para uma arquitetura modular, com vista a resolver outros problemas de computação distribuída.

Referências

- Abdallah, M. and Pucheral, P. (1999). A low-cost non-blocking atomic commitment protocol for asynchronous systems. In *Proceedings of the 11th International Conference on Parallel and Distributed Computing and Systems*, pages 911–918.
- Babaoğlu, O. and Toueg, S. (1993). Non-blocking atomic commitment. In Mullender, S., editor, *Distributed systems*, pages 147–168. 2 edition.
- Chandra, T. D. and Toueg, S. (1996). Unreliable failure detectors for reliable distributed systems. *Journal of the ACM*, 43(2).
- Charron-Bost, B. and Schiper, A. (2004). Uniform consensus is harder than consensus. *Journal of Algorithms*, 51(1):15–37.
- Dettoni, F., Lung, L. C., Correia, M., and Luiz, A. F. (2013). Byzantine fault-tolerant state machine replication with twin virtual machines. In *Proceedings of the 8th Symposium on Computers and Communications*, pages 398–403.
- Dwork, C., Lynch, N. A., and Stockmeyer, L. (1988). Consensus in the presence of partial synchrony. *Journal of ACM*, 35(2):288–322.
- Greve, F. and Narzul, J.-P. L. (2002). Um protocolo de validação atômica não-bloqueante eficiente. In *Anais do XX Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos*, pages 309–323.
- Guerraoui, R. (1995). Revisiting the relationship between non-blocking atomic commitment and consensus. In *Proceedings of the 9th International Workshop on Distributed Algorithms*, pages 87–100.
- Guerraoui, R., Larrea, M., and Schiper, A. (1995). Non blocking atomic commitment with an unreliable failure detector. In *Proceedings of the 14th Symposium on Reliable Distributed Systems*, pages 41–50.
- Guerraoui, R., Larrea, M., and Schiper, A. (1996). Reducing the cost for non-blocking in atomic commitment. In *Proceedings of the 16th International Conference on Distributed Computing Systems*, pages 692–697.
- Guerraoui, R. and Schiper, A. (1995). The decentralized non-blocking atomic commitment protocol. In *Proceedings of the 7th Symposium on Parallel and Distributed Processing*, pages 2–9.
- Hadzilacos, V. (1990). On the relationship between the atomic commitment and consensus problems. In Simons, B. and Spector, A., editors, *Fault-Tolerant Distributed Computing*, volume 448 of *Lecture Notes in Computer Science*, pages 201–208.
- Hadzilacos, V. and Toueg, S. (1994). A modular approach to the specification and implementation of fault-tolerant broadcasts and related problems. Technical Report TR 94-1425, Department of Computer Science, Cornell University.
- Stumm Júnior, V., Lung, L. C., Correia, M., Fraga, J. S., and Lau, J. (2010). Intrusion tolerant services through virtualization: A shared memory approach. In *Proceedings of the 24th International Conference on Advanced Information Networking and Applications*, pages 768–774.
- Lamport, L., Shostak, R., and Pease, M. (1982). The Byzantine generals problem. *ACM Transactions on Programming Languages and Systems*, 4(3):382–401.
- Luiz, A. F., Lung, L. C., Correia, M., and Stumm Júnior, V. (2013). Validação atômica não-bloqueante com faltas bizantinas. Relatório Técnico, PGEAS/DAS/UFSC. Disponível em <http://www.das.ufsc.br/~aldelir/report2013-01.pdf>.
- Park, S.-H., Lee, J.-Y., and Yu, S.-C. (2013). Non-blocking atomic commitment algorithm in asynchronous distributed systems with unreliable failure detectors. In *Proceedings of the 10th International Conference on Information Technology*, pages 33–38.
- Pease, M., Shostak, R., and Lamport, L. (1980). Reaching agreement in the presence of faults. *Journal of ACM*, 27(2):228–234.
- Popek, G. J. and Goldberg, R. P. (1974). Formal requirements for virtualizable third generation architectures. *Communications of the ACM*, 17(7):412–421.
- Rivest, R. L., Shamir, A., and Adleman, L. M. (1978). A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126.
- Shoup, V. (2000). Practical threshold signatures. In *Proceedings of the 19th International Conference on Theory and Application of Cryptographic Techniques*, pages 207–220.
- Smith, J. E. and Nair, R. (2005). The architecture of virtual machines. *Computer*, 38(5):32–38.